University of Nottingham
UK | CHINA | MALAYSIA

COMP4139 Machine Learning (MLE)

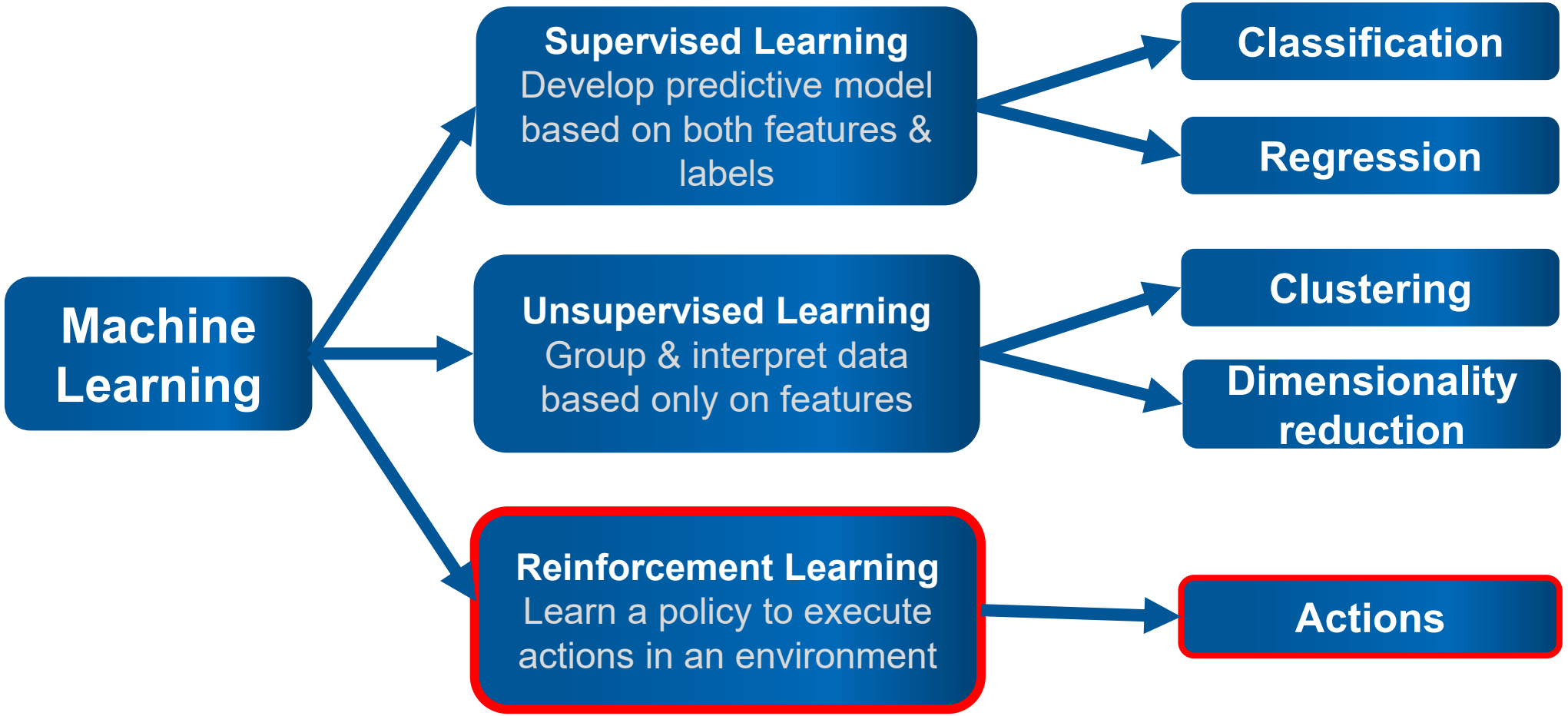
Reinforcement Learning

Dr Shreyank N Gowda
Dr Direnc Pekaslan

School of Computer Science
University of Nottingham

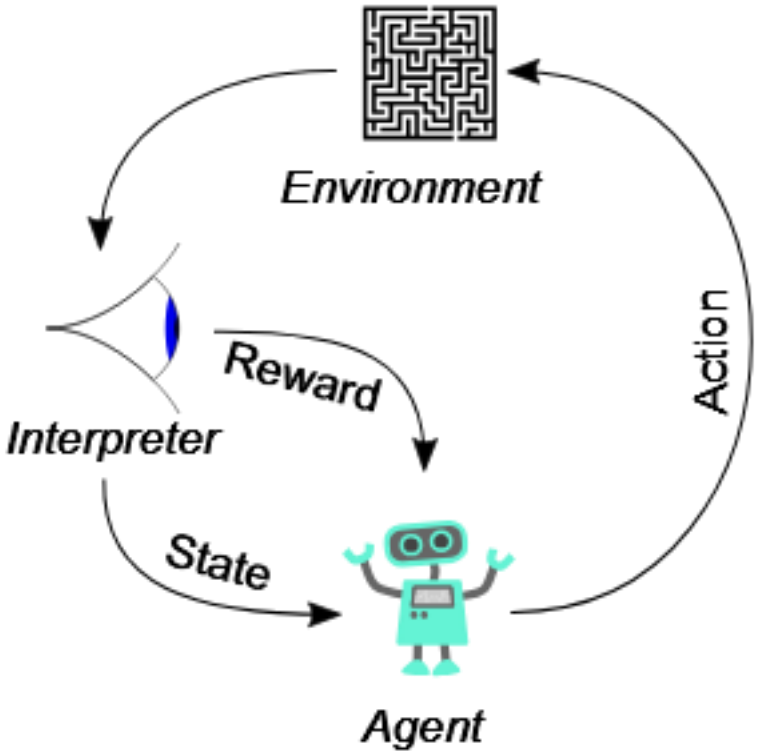
Machine Learning

- Train machines to explore useful features and learn data patterns to predict/infer a target event.



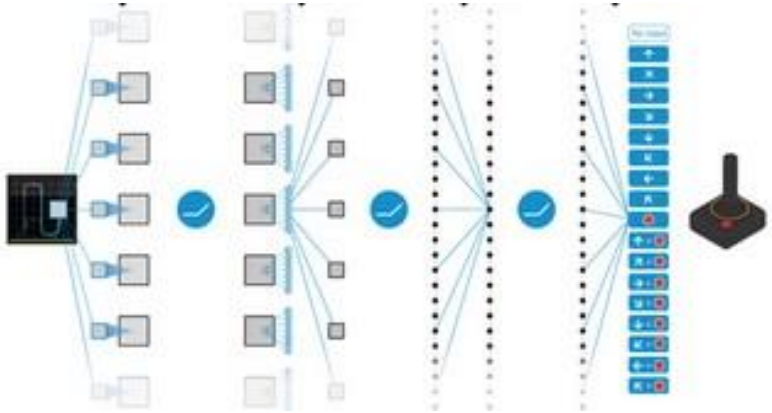
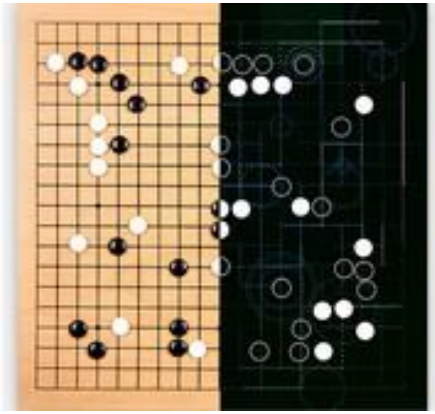
Reinforcement Learning

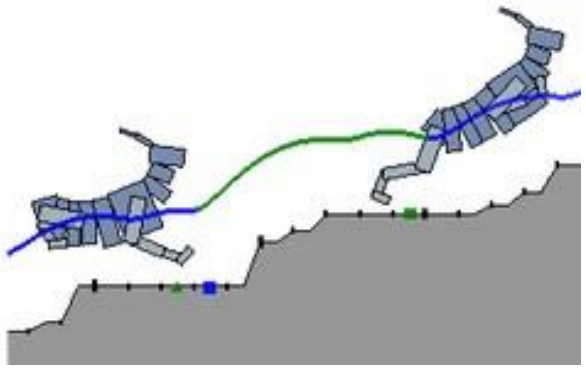
- RL trains an agent to take actions in an environment by sending reward and state.
  - Employs trial and error to find a solution
  - Make a sequence of decisions
  - Do not require labelled input/output pairs but rules of reward and penalty.
  - Aim to take actions by maximising reward.



Reinforcement Learning

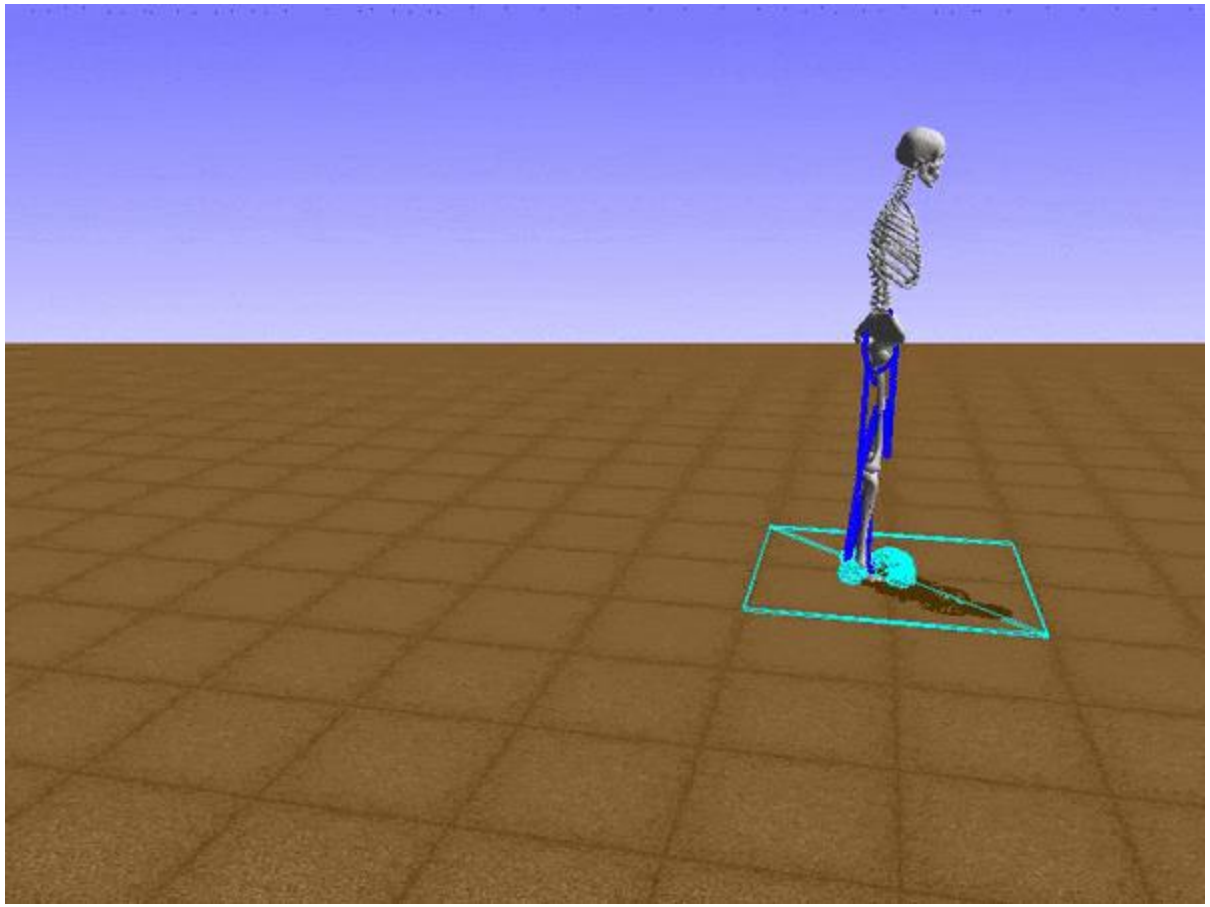
- RL is studied and applied in broad application areas.
  - Game
  - Robotics
  - Logistics
  - Autonomous driving

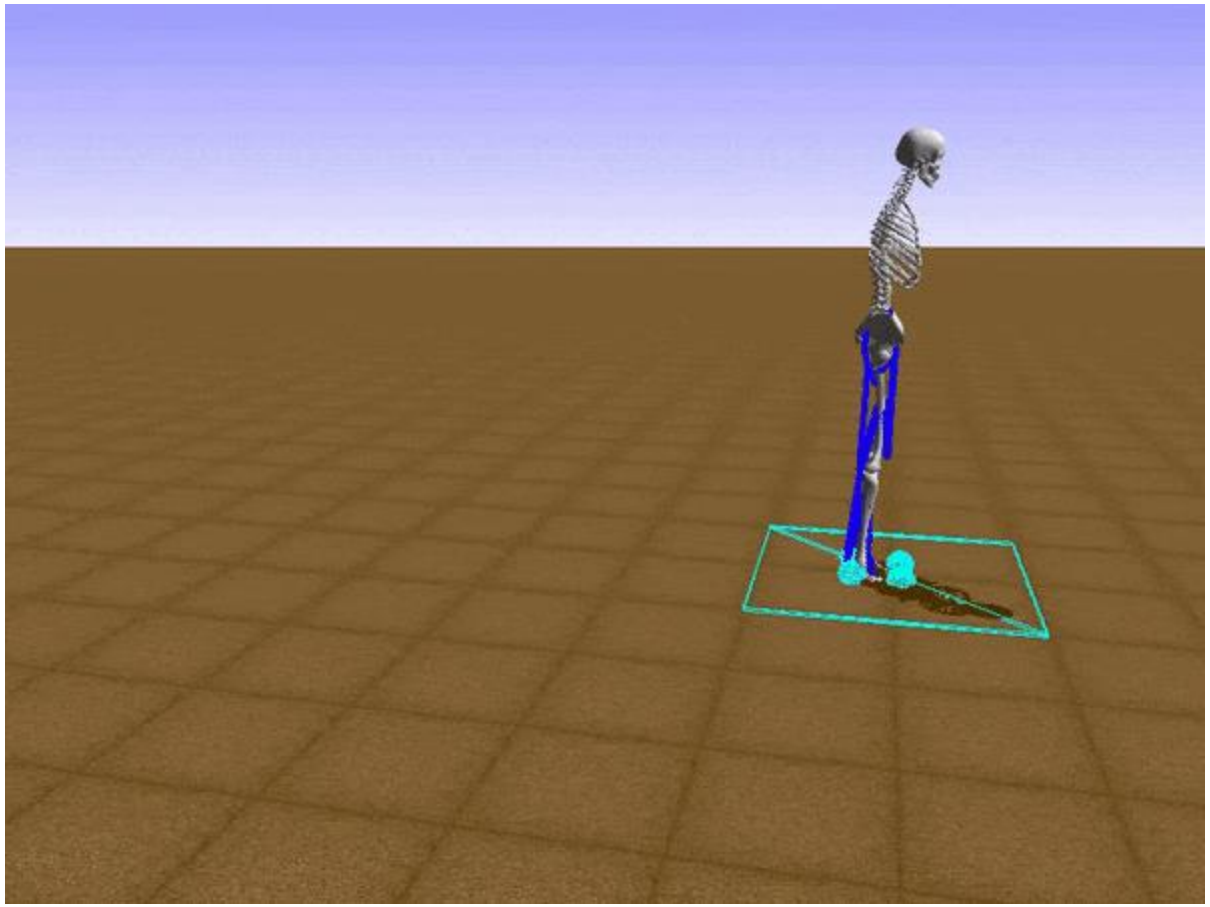


A nice example (learn to run): https://deepsense.ai/learning-to-run-an-example-of-reinforcement-learning/

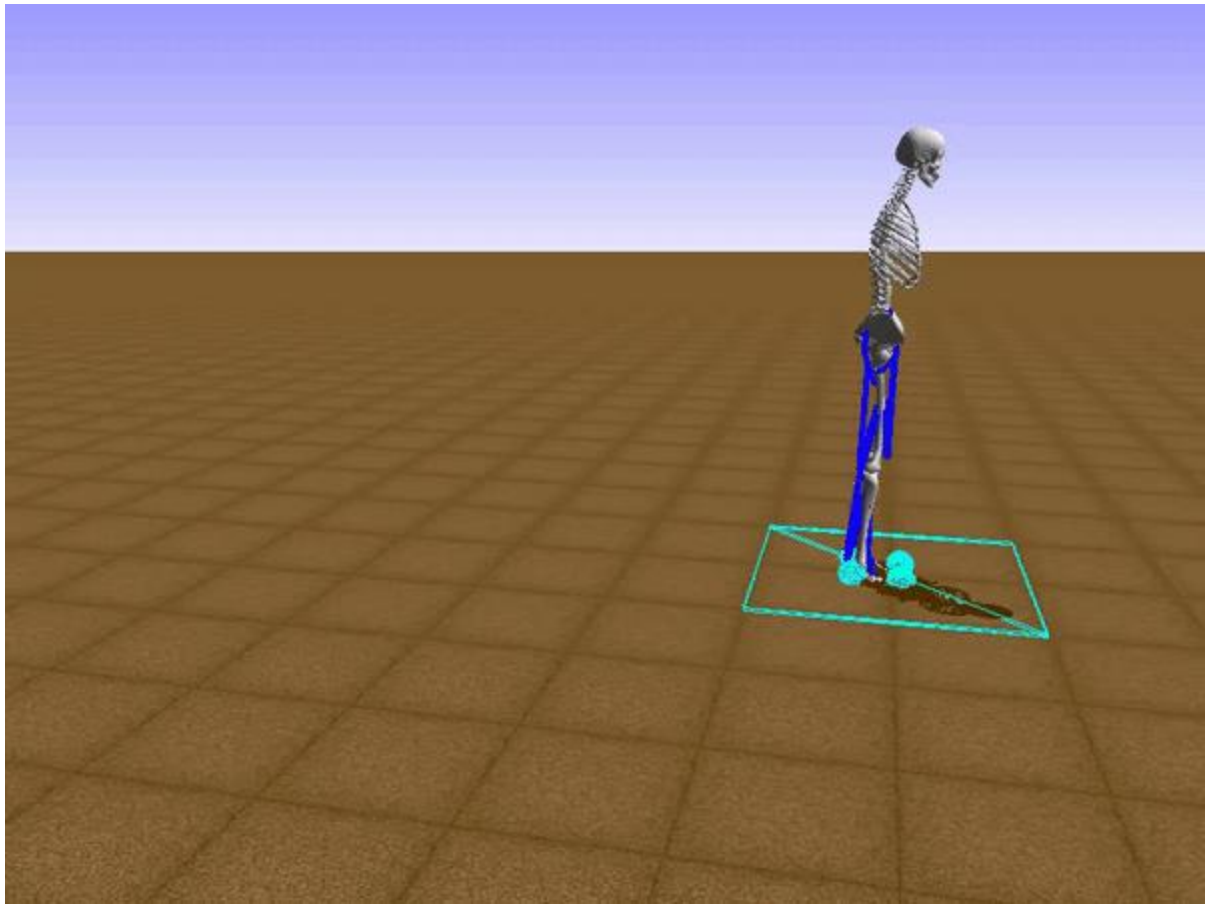
Example



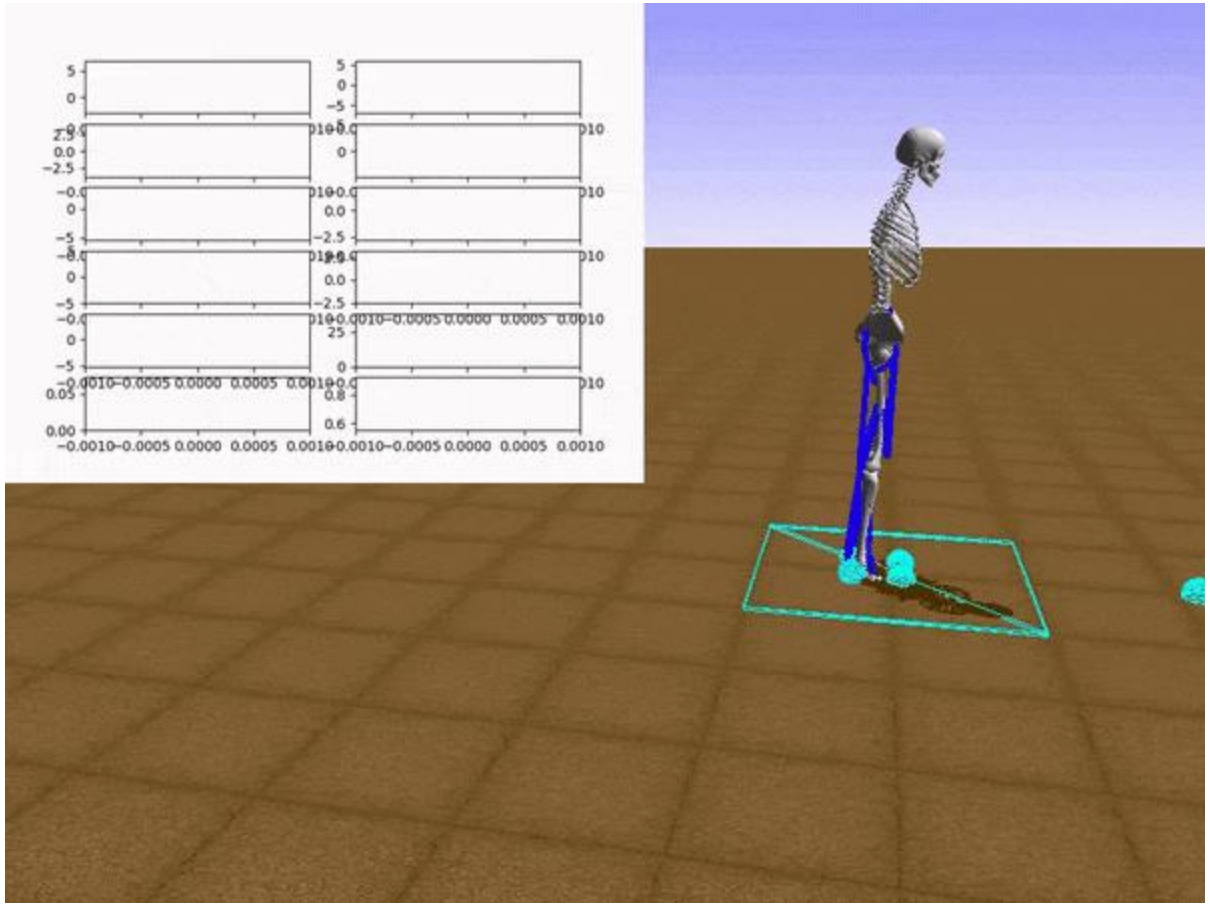
Example



Example

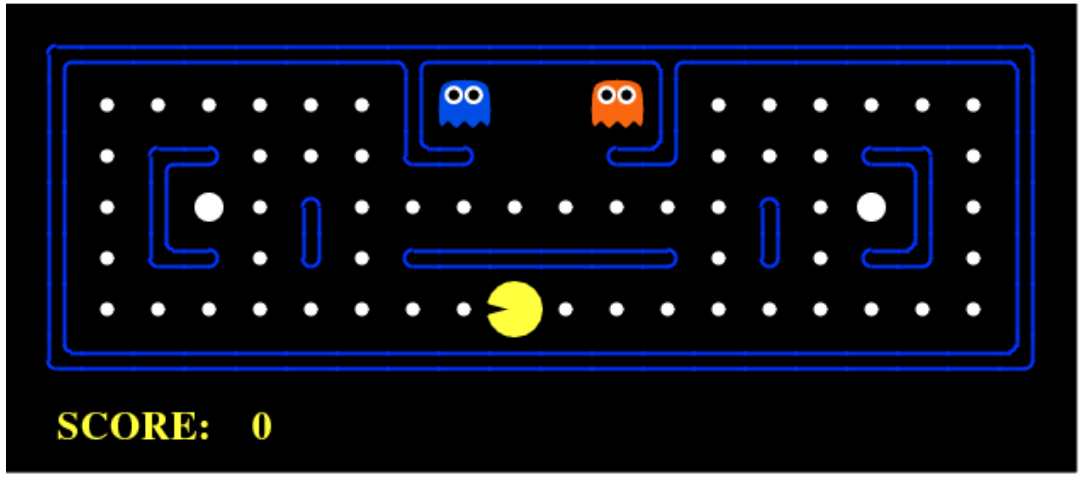


Example



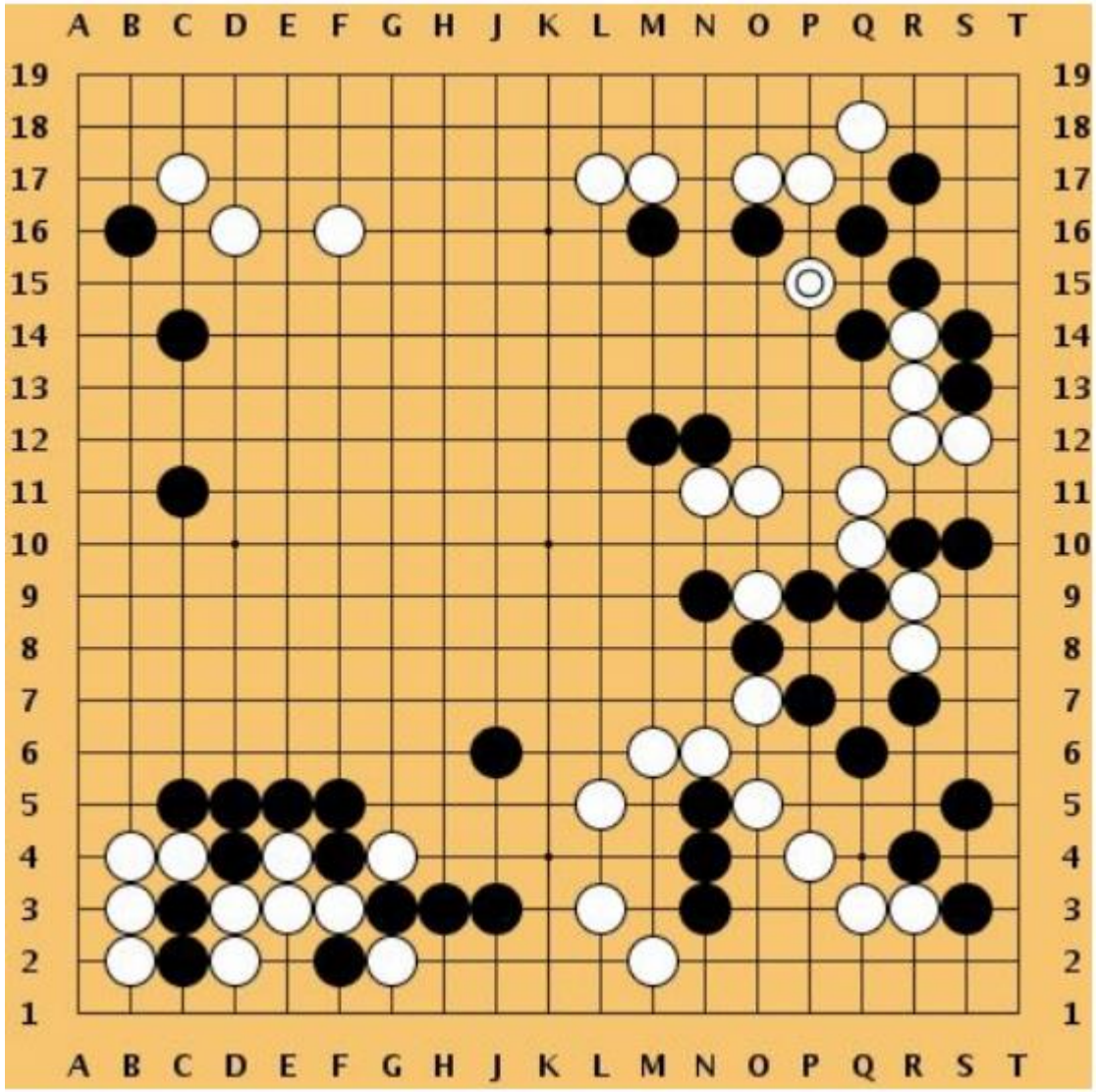
Key Elements of RL

- Environment
  - Physical world in which the agent operates
- Agent
  - Learn to act that maximises cumulative reward
- State
  - Current situation of the agent
- Reward
  - Feedback from the environment
- Policy
  - Method to map agent's state to actions
- Value
  - Future reward that an agent would receive by taking an action in a particular state



Environment?	Grid map
Agent?	PacMan
State?	Location of the agent
Reward?	+:Eat pea; - get caught
Policy?	Rules to move
Value?	Future reward for an action
Aim?	Learn an optimal policy to maximise cumulative reward

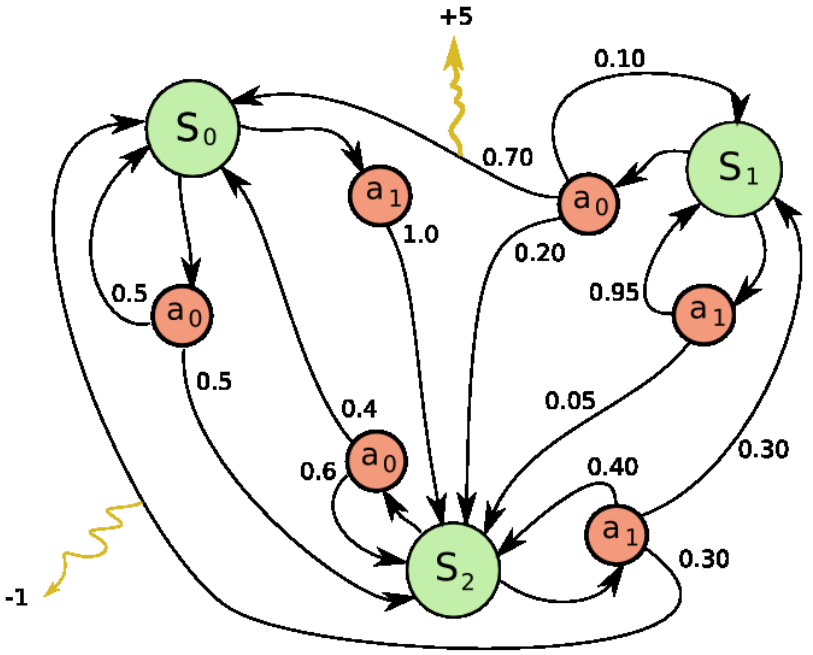
Game of Go



Environment?	Game Board
Agent?	AI player to place the piece
State?	Position of all pieces
Reward?	1 win at the end, 0 otherwise
Policy?	Rules to place the piece
Value?	Future reward for an action
Aim?	Win the game

Key Element of Markov Decision Process

- Construct a set of environment states S
- Define a set of possible actions A
- Define a real valued reward function R
- Build a transition model P(s', s|a).
- Hyperparameter r as the discount factor.



We assume the Markov Property: *the effects of an action taken in a state depend only on that state and not on the prior history.*

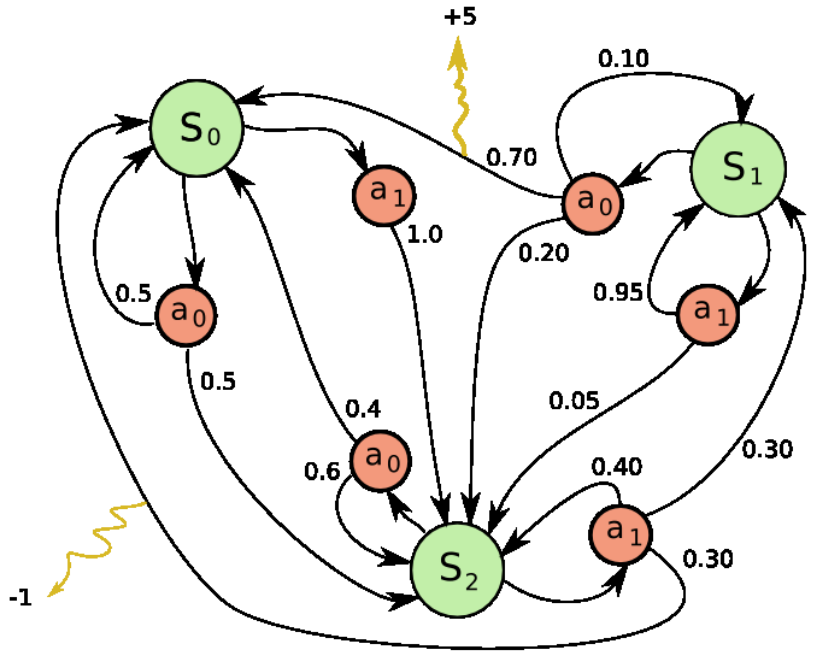
Example of MDP: three states S0, S1 and S2. Two actions a0, a1, and two rewards.(+5,-1)

Aim of Markov Decision Process

- Find a good "policy" for the Agent to act (a) at state S.
- This good “policy” maximises a cumulative reward (expected return).

$$G_t = R_{t+1} + \gamma R_{t+2} + \cdots = \sum_{k=0}^{\infty} r^k R_{t+k+1}$$

- Optimisation methods to solve it (e.g. dynamic programming).



Example of MDP: three states S0, S1 and S2. Two actions a0, a1, and two rewards.(+5,-1)

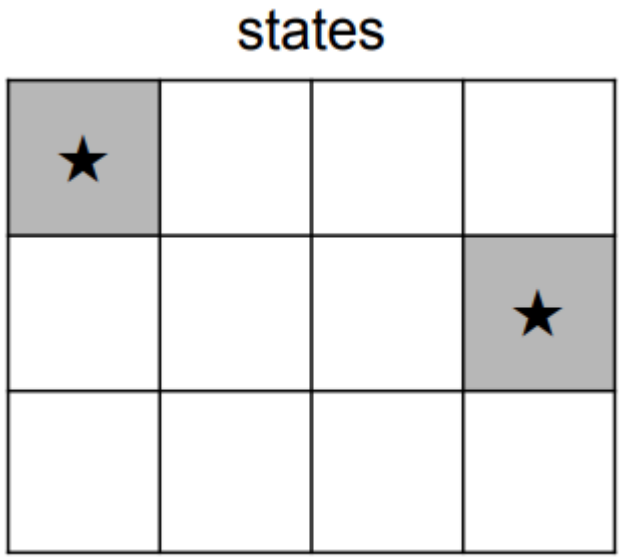
Learning Process of Markov Decision Process

- At time step t=0, environment samples initial state S_0 from $p(S_0)$
- For t=0 until done:
  - Agent selects action a_t
  - Environment samples reward $r_t \sim R(.|s_t,a_t)$
  - Environment samples the next state $s_{t+1} \sim P(.|s_t,a_t)$
  - Agent receives reward r_t and next state s_{t+1}
- A policy is a function from S to A that specifies what action to take in each state.
- Objective: find the best policy that maximises the cumulative discounted reward.

Example of MDP

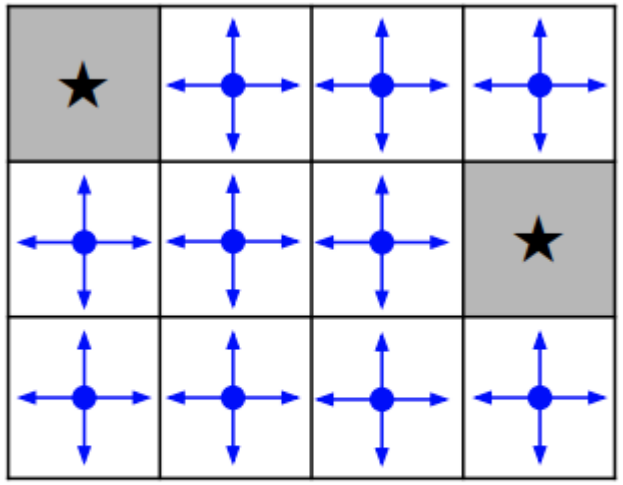
- Objective: reach one of the stared position with minimum steps
- Reward: -1 for each transition

actions = {
1. right
2. left
3. up
4. down
}

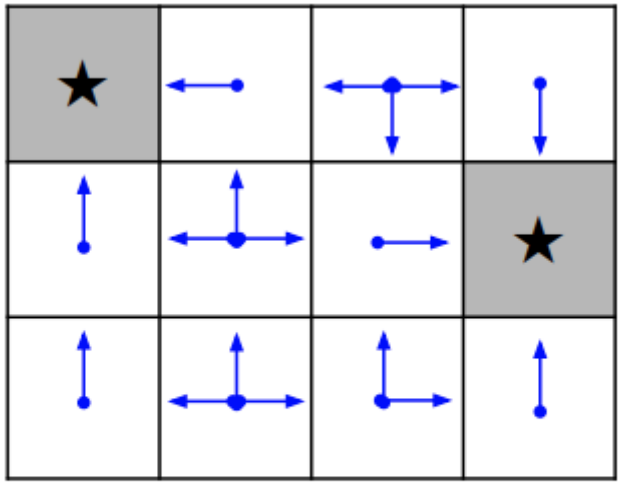


Example of MDP

- Find the optimal policy that maximise the reward.



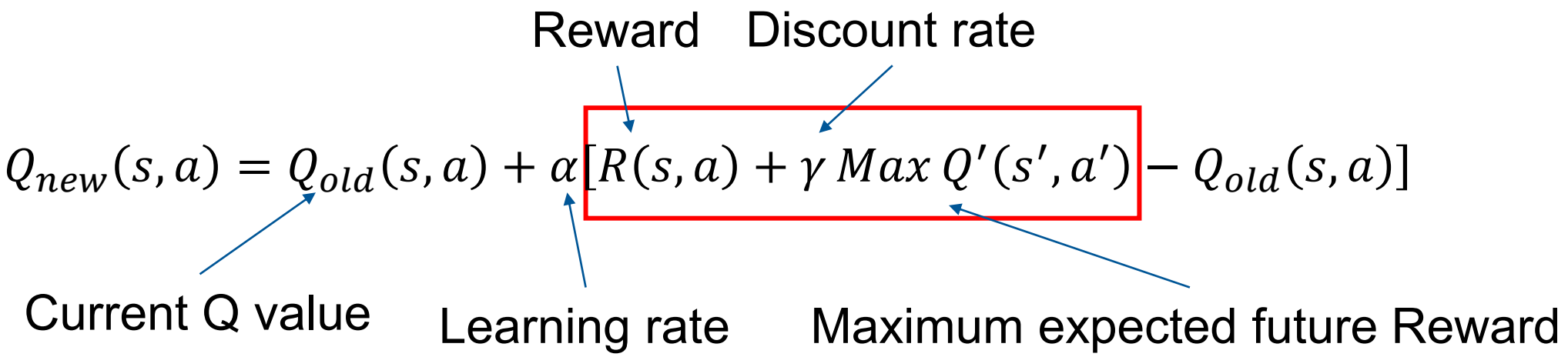
Random Policy



Optimal Policy

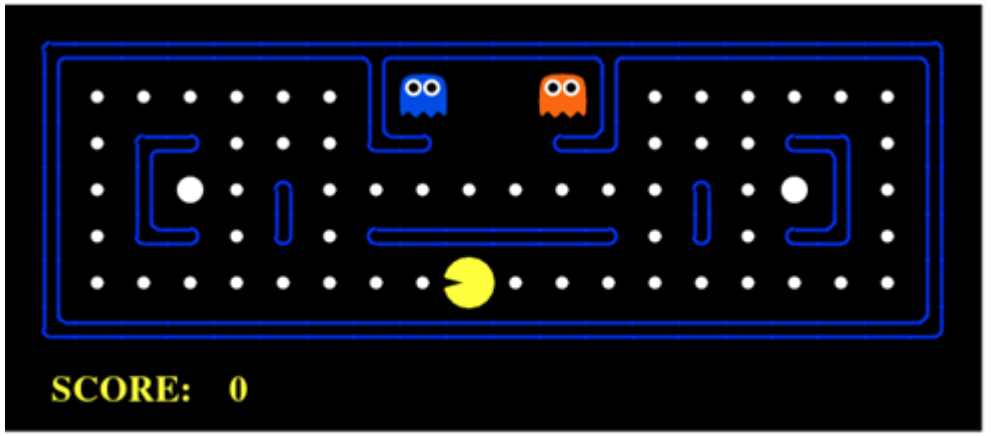
Q-Learning

- Q-learning is a method to find the next best action (a) given a current state (s), Q(s,a) that aims to maximise cumulative reward.
- Q value defines the quality of State/Action pair
- Bellman Equation: determine the value of a particular state and deduce how good it is to be in that state.



Q-Learning Process

- Initialise Q-values in a Q-table (e.g. 0s)
- Choose an action ***a*** for state ***s*** (best Q-value)
- Perform action ***a***, results in a new state ***s'***
- Measure reward ***R***
- Update ***Q*** with Bellman Equation



$$Q_{new}(s,a) = Q_{old}(s,a) + \alpha[R(s,a) + \gamma\, Max\, Q'(s',a') - Q_{old}(s,a)]$$

Reward

Discount rate

Current Q value

Learning rate

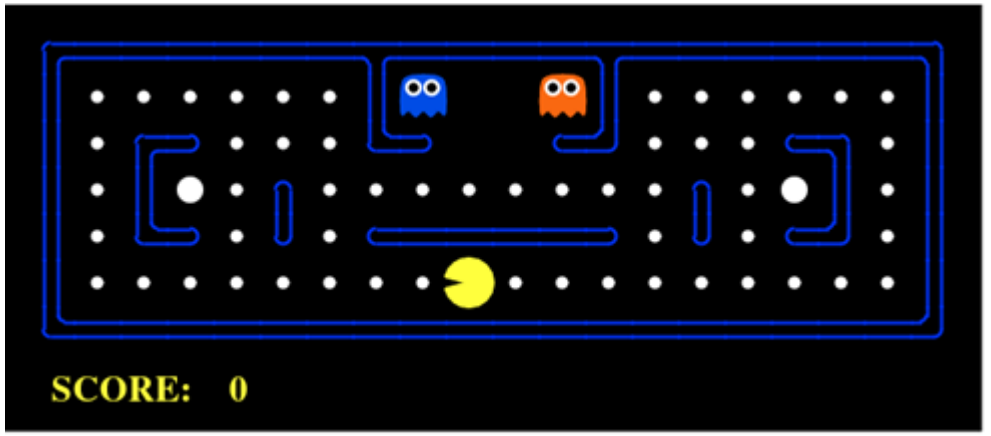
Maximum expected future Reward

Q Table

States\Actions	UP	Down	Left	Right
Both Ghosts at top				
Both Ghosts at right				
One left one right				
….				

Q-Learning Process

- **Initialise Q-values in a Q-table (e.g. 0s)**
- Choose an action ***a*** for state ***s*** (best Q-value)
- Perform action ***a***, results in a new state ***s'***
- Measure reward ***R***
- Update ***Q*** with Bellman Equation



$$Q_{new}(s,a) = Q_{old}(s,a) + \alpha[R(s,a) + \gamma\, Max\, Q'(s',a') - Q_{old}(s,a)]$$

Reward

Discount rate

Current Q value

Learning rate

Maximum expected future Reward

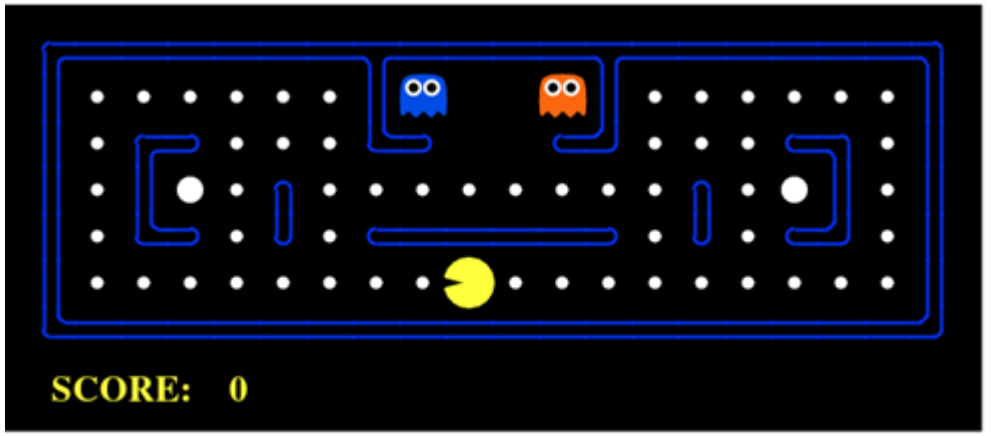
Q Table

States\Actions	UP	Down	Left	Right
Both Ghosts at top	0	0	0	0
Both Ghosts at right	0	0	0	0
One left one right	0	0	0	0
….				

University of Nottingham
UK | CHINA | MALAYSIA

Q-Learning Process

- Initialise Q-values in a Q-table (e.g. 0s)
- **Choose an action *a* for state *s* (best Q-value)**
- Perform action ***a***, results in a new state ***s'***
- Measure reward ***R***
- Update ***Q*** with Bellman Equation



$$Q_{new}(s,a) = Q_{old}(s,a) + \alpha[R(s,a) + \gamma\, Max\, Q'(s',a') - Q_{old}(s,a)]$$

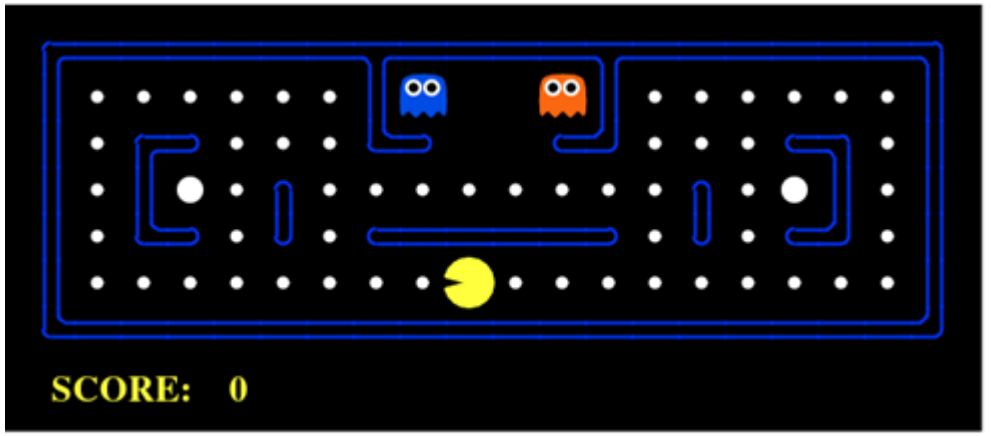
Reward
Discount rate
Current Q value
Learning rate
Maximum expected future Reward

Q Table

States\Actions	UP	Down	Left	Right
Both Ghosts at top	0	0	0	0
Both Ghosts at right	0	0	0	0
One left one right	0	0	0	0
….				

Q-Learning Process

- Initialise Q-values in a Q-table (e.g. 0s)
- Choose an action *a* for state *s* (best Q-value)
- **Perform action *a*, results in a new state *s'***
- Measure reward ***R***
- Update ***Q*** with Bellman Equation



$$Q_{new}(s,a) = Q_{old}(s,a) + \alpha[R(s,a) + \gamma\, Max\, Q'(s',a') - Q_{old}(s,a)]$$

Reward

Discount rate

Current Q value

Learning rate

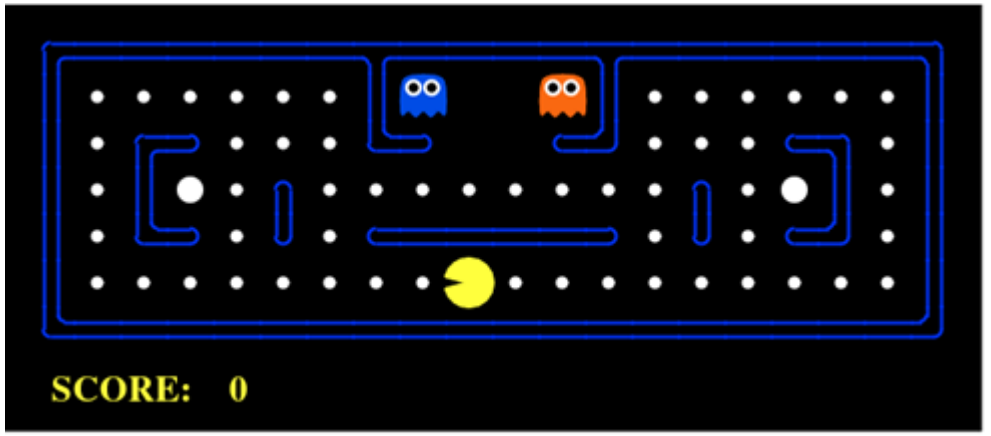
Maximum expected future Reward

Q Table

States\Actions	UP	Down	Left	Right
Both Ghosts at top	0	0	0	0
Both Ghosts at right	0	0	0	0
One left one right	0	0	0	0
….				

Q-Learning Process

- Initialise Q-values in a Q-table (e.g. 0s)
- Choose an action *a* for state *s* (best Q-value)
- Perform action *a*, results in a new state *s'*
- **Measure reward *R***
- Update ***Q*** with Bellman Equation



$$Q_{new}(s,a) = Q_{old}(s,a) + \alpha[R(s,a) + \gamma\, Max\, Q'(s',a') - Q_{old}(s,a)]$$

Reward

Discount rate

Current Q value

Learning rate

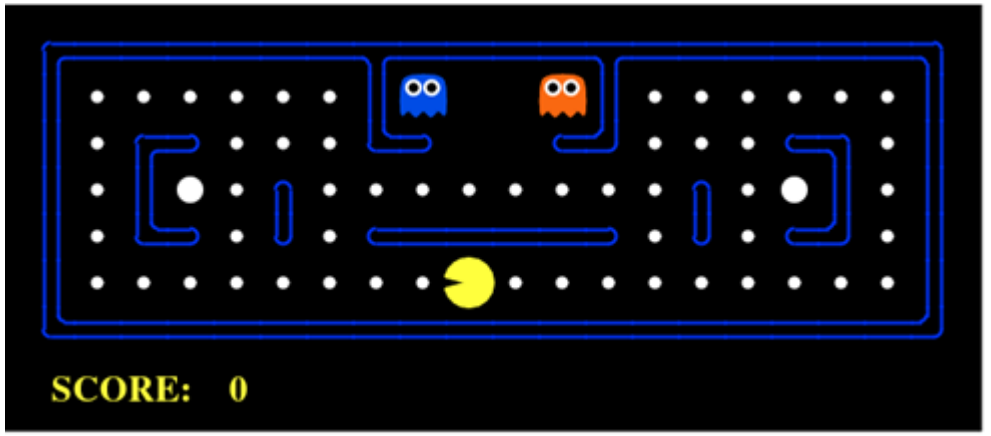
Maximum expected future Reward

Q Table

States\Actions	UP	Down	Left	Right
Both Ghosts at top	0	0	0	0
Both Ghosts at right	0	0	0	0
One left one right	0	0	0	0
….				

Q-Learning Process

- Initialise Q-values in a Q-table (e.g. 0s)
- Choose an action *a* for state *s* (best Q-value)
- Perform action *a*, results in a new state *s'*
- Measure reward *R*
- **Update *Q* with Bellman Equation**



$$Q_{new}(s,a) = Q_{old}(s,a) + \alpha[R(s,a) + \gamma\, Max\, Q'(s',a') - Q_{old}(s,a)]$$

Reward

Discount rate

Current Q value

Learning rate

Maximum expected future Reward

Q Table

States\Actions	UP	Down	Left	Right
Both Ghosts at top	10	0	0	0
Both Ghosts at right	0	0	0	0
One left one right	0	0	0	0
….				

Q-Learning Process

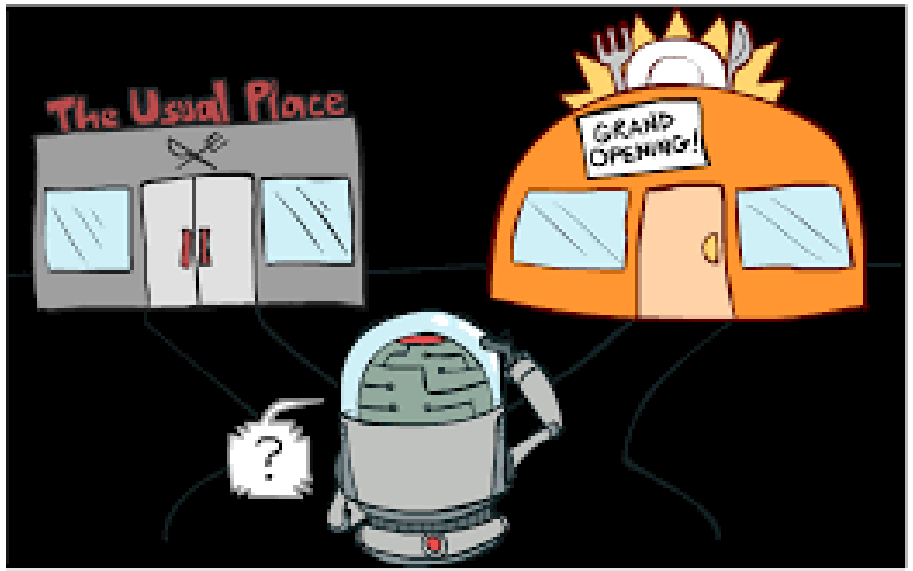
- Keep updating the Q table by playing the game many times.
- Once completed training, use Q table for taking actions by choose the highest value.
- Drawback: computationally expensive in both training and inference

Q Table

States\Actions	UP	Down	Left	Right
Both Ghosts at top	10	30	40	10
Both Ghosts at right	0	100	1000	50
One left one right	50	50	-100	-100
….				

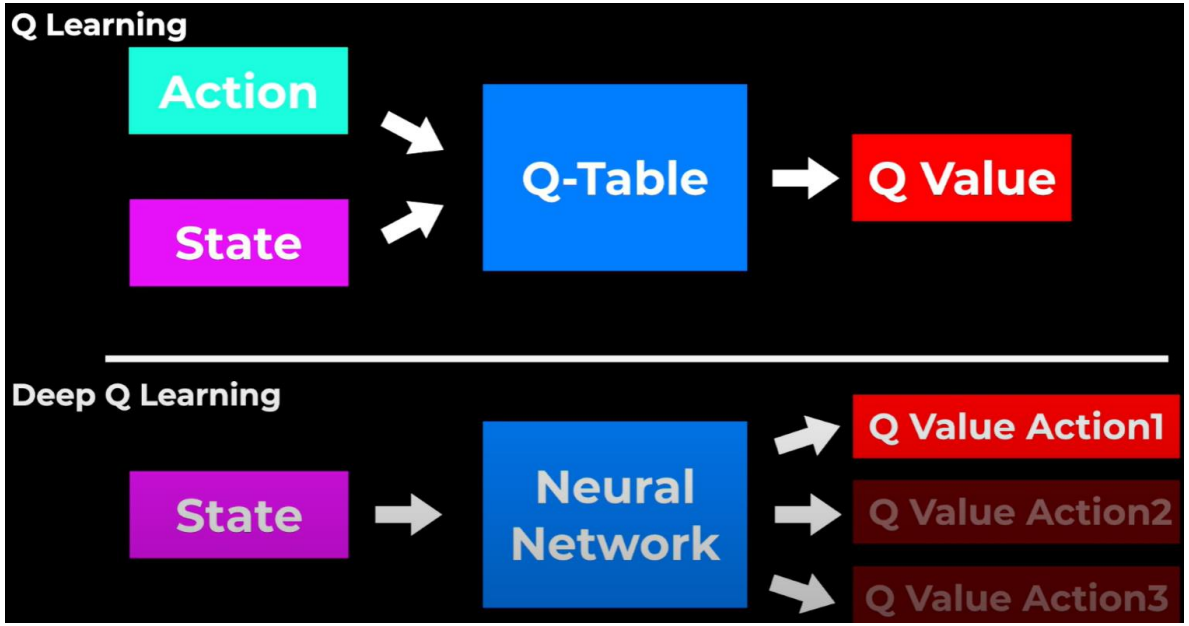
Exploration vs Exploitation

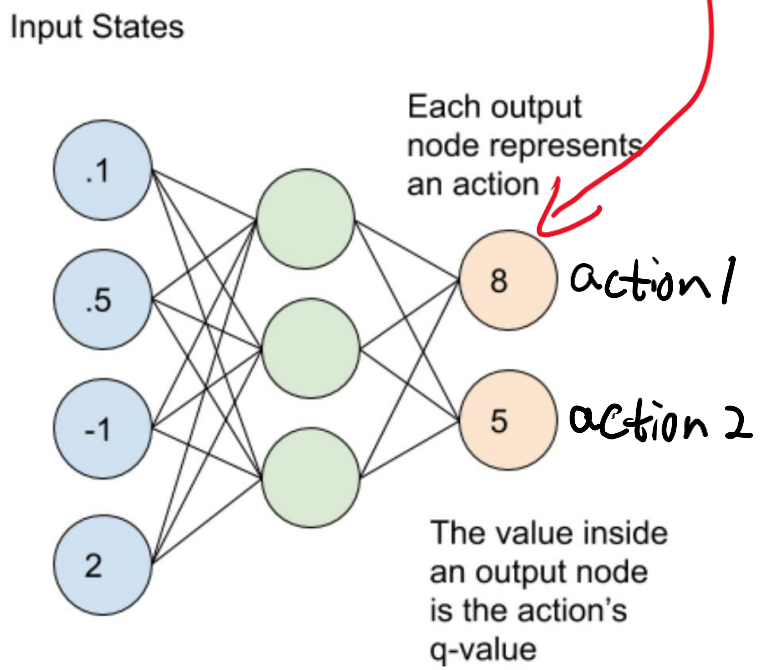
- Agent don't normally know the environment.
  - Introduce some randomness on selecting actions
- Epsilon-Greedy Exploration Strategy
  - At every time step when it's time to choose an action, roll a dice
  - If it has a probability less than epsilon, choose a random action
  - Otherwise take the best-known action at the agent's current state



Deep Q-Learning (DQN)

- DQN: a neural network maps input states to action/Q-value pair.
  - Initialise Q-values (network)
  - Network predict an action ***a*** for state ***s***
  - Perform action ***a***, results in a new state ***s'***
  - Measure reward ***R***
  - Update weights of Neural Network using the Bellman Equation. $R(s,a)+\gamma \max Q(s',a')$

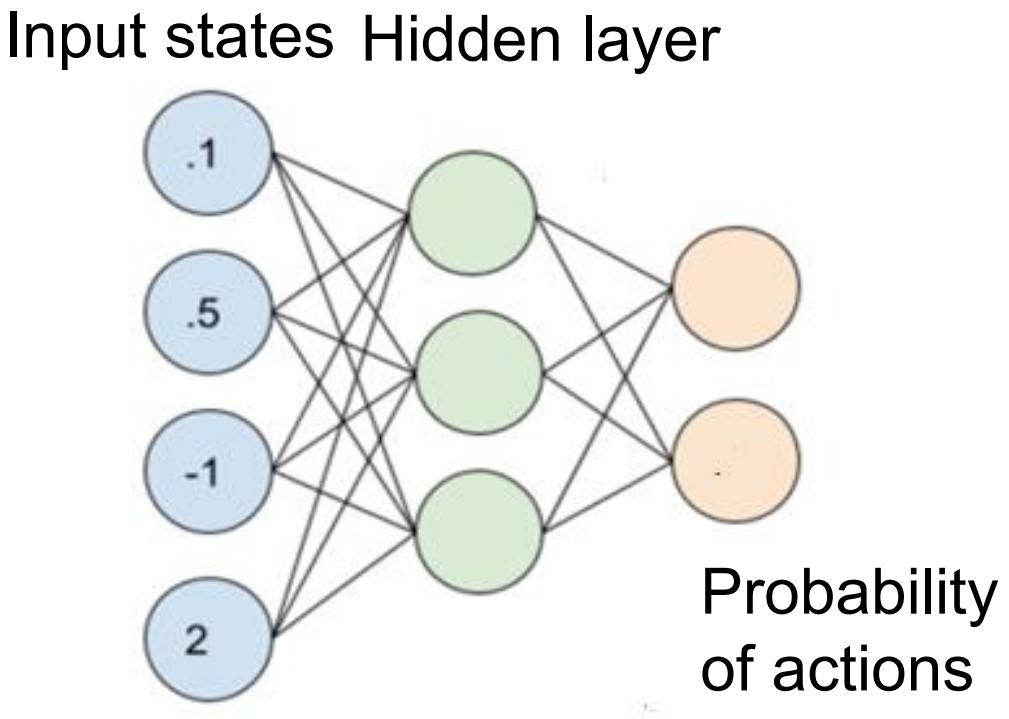




Example: https://github.com/mswang12/minDQN/blob/main/minDQN.py

Policy Gradients (PG)

- PG uses a policy network to directly estimate actions rather than quality values
- Input states (image, coordinates), output actions
- Train the Policy network using policy gradient. How?



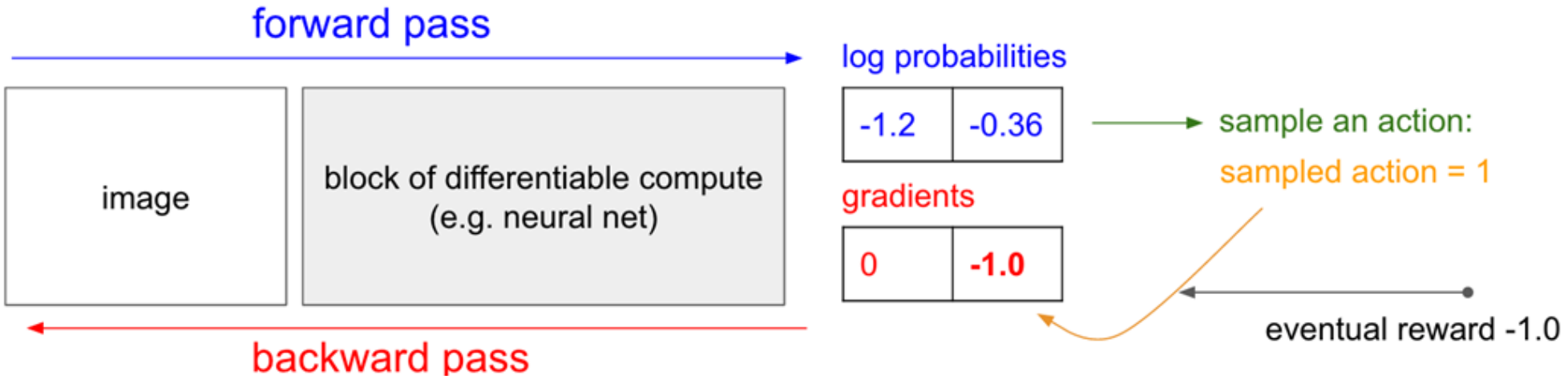
http://karpathy.github.io/2016/05/31/rl/

Training Policy Network

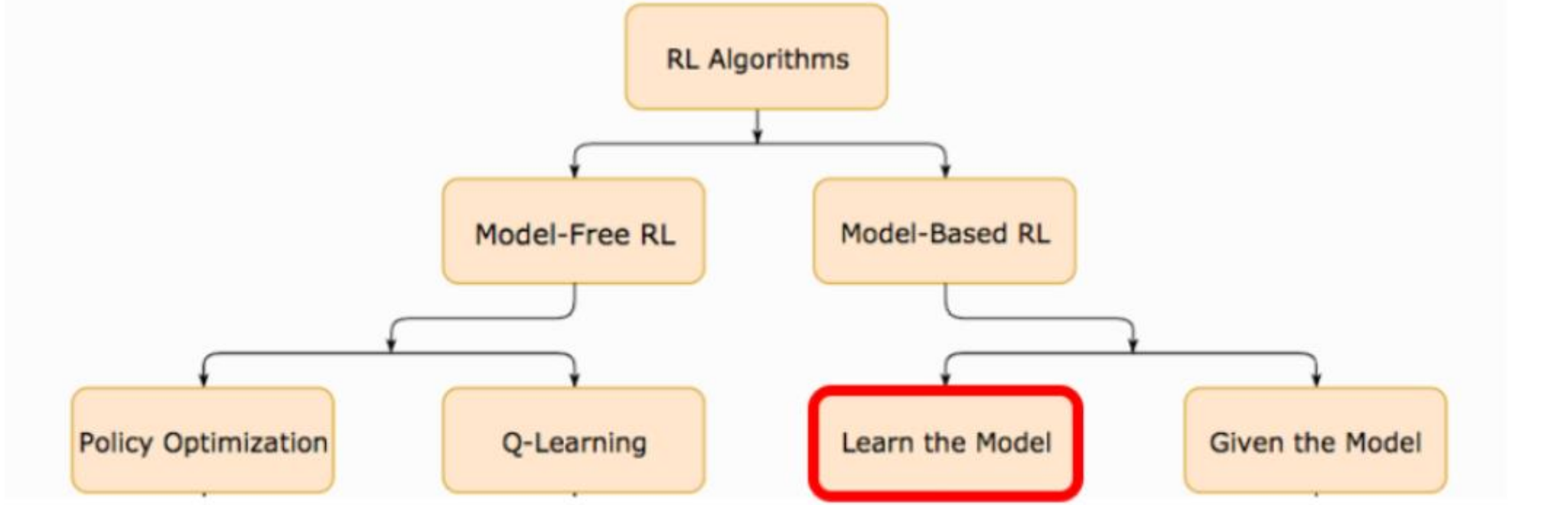
- Initialise the network with random weights
- Forward passing the network to generate a possible action
- With this action, keep executing the game until end (e.g. game over).
  - For a positive reward, encourage the selected action by backpropagating a positive gradient
  - For a negative reward, discourage the selected action by backpropagating a negative gradient
- Play *N* episodes (e.g. rounds of games), updates the weights based on positive/negative gradients.

Use discounted reward

$$\sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$



Model-free vs Model-based



$\pi_\theta(\boldsymbol{a}_t|\boldsymbol{s}_t)$

- Easier to implement and tune
- More popular and have been more extensively developed and tested [1]
- Need More Samples

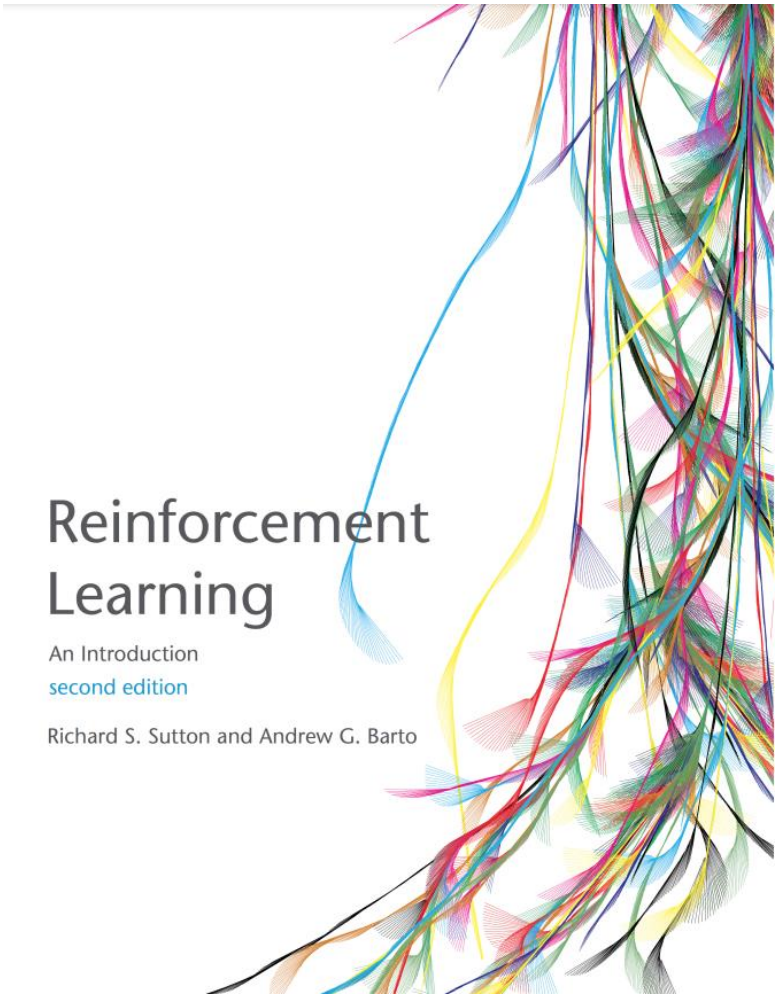
$p(\boldsymbol{s}_{t+1}|\boldsymbol{s}_t, \boldsymbol{a}_t)$ $\quad f(\boldsymbol{s}_t, \boldsymbol{a}_t) = \boldsymbol{s}_{t+1}$

- Need Less Samples
- Allow the agent to plan by thinking ahead (AlphaZero)
- Complex optimization methods
- More assumptions and approximations

Summary and Challenges in Using RL

- Model free method is the way forward and more flexible in dealing with unknown environment but slow to train.
- Policy gradients normally performed better than deep Q learning.
- Requires large data to train, as no direct controlling of the agent but only through rewarding. Even more data dependent than DL.
- Reaching a local optimum: performing not as expected but the agent thinks it's doing well.
- The agent finds a shortcut in getting the rewards without completing the designed task.

Learn More about RL



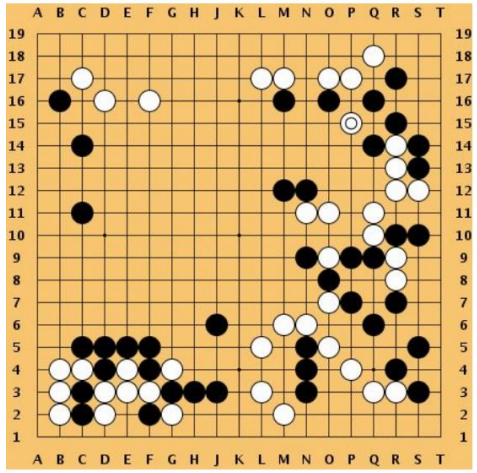
Pixels to Pong:
https://gist.github.com/karpathy/a4166c7fe253700972fcbc77e4ea32c5

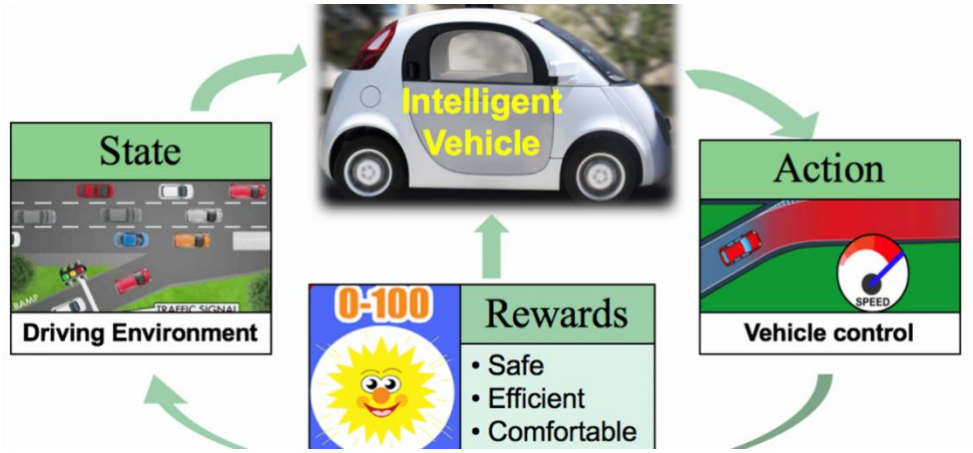
OpenAI Gym:
https://www.gymlibrary.dev/

http://incompleteideas.net/book/the-book-2nd.html

Popular Applications

- Alpha Go: supervised learning and reinforcement learning. Monte Carlo Tree Search and deep RL.
- Autonomous Driving: understand the driving environment based on computer vision; design effective rewards.
- Humanoid Robot: deep learning, computer vision, motion planning, controls, mechanical.





[Mastering the game of Go with deep neural networks and tree search (storage.googleapis.com)]